Федеральное государственное автономное образовательное учреждение высшего образования Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники

Сервис-ориентированная архитектура
Лабораторная работа №1
Вариант № 67609

Выполнили: студенты группы P3408,
Васильев Н. А., Пронкин А. Д.

Преподаватель: Кривоносов Е. Д.

Санкт-Петербург 2026

Текст задания

Введите вариант:

67609

Внимание! У разных вариантов разный текст задания!

Разработать спецификацию в формате OpenAPI для набора веб-сервисов, реализующего следующую функциональность:

Первый веб-сервис должен осуществлять управление коллекцией объектов. В коллекции необходимо хранить объекты класса `Organization`, описание которого приведено ниже:

```
public class Organization {
    private long id; //Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private Double annualTurnover; //Поле может быть null, Значение поля должно быть больше 0
    private OrganizationType type; //Поле может быть null
    private Address officialAddress; //Поле может быть null
}

public class Coordinates {
    private long x;
    private int y;
}

public class Address {
    private String zipCode; //Длина строки не должна быть больше 18, Поле не может быть null
    private Location town; //Поле может быть null
}

public class Location {
    private long x;
    private long y;
    private String name; //Поле не может быть null
}

public enum OrganizationType {
    PUBLIC,
    GOVERNMENT,
    TRUST,
    PRIVATE_LIMITED_COMPANY;
}
```

Веб-сервис должен удовлетворять следующим требованиям:

- API, реализуемый сервисом, должен соответствовать рекомендациям подхода RESTful.
- Необходимо реализовать следующий базовый набор операций с объектами коллекции: добавление нового элемента, получение элемента по ИД, обновление элемента, удаление элемента, получение массива элементов.
- Операция, выполняемая над объектом коллекции, должна определяться методом HTTP-запроса.
- Операция получения массива элементов должна поддерживать возможность сортировки и фильтрации по любой комбинации полей класса, а также возможность постраничного вывода результатов выборки с указанием размера и порядкового номера выводимой страницы.
- Все параметры, необходимые для выполнения операции, должны передаваться в URL запроса.
- Информация об объектах коллекции должна передаваться в формате xml.
- В случае передачи сервису данных, нарушающих заданные на уровне класса ограничения целостности, сервис должен возвращать код ответа http, соответствующий произошедшей ошибке.

Помимо базового набора, веб-сервис должен поддерживать следующие операции над объектами коллекции:

- Удалить один (любой) объект, значение поля type которого эквивалентно заданному.
- Вернуть количество объектов, значение поля type которых больше заданного.
- Вернуть массив объектов, значение поля annualTurnover которых меньше заданного.

Эти операции должны размещаться на отдельных URL.

Второй веб-сервис должен располагаться на URL `/orgmanager`, и реализовывать ряд дополнительных операций, связанных с вызовом API первого сервиса:

- `/hire/{id}` : добавить нового сотрудника в организацию с указанным id
- `/acquise/{acquirer-id}/{acquired-id}` : реализовать поглощение одной организацией другой без увольнения сотрудников

Эти операции также должны размещаться на отдельных URL.

Для разработанной спецификации необходимо сгенерировать интерактивную веб-документацию с помощью Swagger UI. Документация должна содержать описание всех REST API обоих сервисов с текстовым описанием функциональности каждой операции. Созданную веб-документацию необходимо развернуть на сервере `helios`.

Код

Ссылка на GitHub: https://github.com/ren1kron/soa-lab1

Инструкции по настройке, деплою и развертыванию лабораторной работы

Для локального запуска необходимо открыть терминал в корневом каталоге проекта и запустить статический HTTP-сервер:

```
python3 -m http.server 26125 --bind 127.0.0.1
```

После запуска документация будет доступна в браузере по адресу:

```
http://127.0.0.1:26125/
```

HTTP-сервер необходимо оставить запущенным на всё время работы с документацией.

По умолчанию первый сервис доступен по адресу:

```
http://localhost:8080
```

Второй сервис:

```
http://localhost:8081
```

Для размещения документации на Helios необходимо скопировать `index.html`, `openapi.yaml`, `swagger.css` и каталог `vendor/` на удалённый сервер.

На локальном компьютере из корневого каталога проекта выполняются команды:

```
ssh helios 'mkdir -p ~/soa/soa-lab1/dist'
scp -r index.html openapi.yaml swagger.css vendor helios:~/soa/soa-lab1/dist/
```

Первая команда создаёт необходимый каталог на Helios, если он ещё не существует. Вторая команда копирует файлы документации на удалённый сервер.

После загрузки необходимо подключиться к Helios:

```
ssh helios
```

Перейти в каталог документации:

```
cd ~/soa/soa-lab1/dist
```

И запустить статический HTTP-сервер:

```
python -m http.server 26125 --bind 127.0.0.1
```

Если на порту `26125` уже работает ранее запущенный сервер, его необходимо предварительно остановить комбинацией `Ctrl+C`.

Поскольку HTTP-сервер на Helios привязан к адресу `127.0.0.1`, для доступа к документации с локального компьютера используется SSH-туннель.

Во втором локальном терминале необходимо выполнить:

```
ssh -N -o ExitOnForwardFailure=yes -L 26125:127.0.0.1:26125 helios
```

После создания туннеля документация, работающая на Helios, становится доступна на локальном компьютере по адресу:

```
http://127.0.0.1:26125/
```

Во время работы необходимо оставить открытыми оба SSH-соединения: терминал с запущенным HTTP-сервером и терминал с SSH-туннелем.

Вывод

В ходе выполнения работы была разработана спецификация REST API двух веб-сервисов в формате OpenAPI. Были изучены принципы описания HTTP-операций, параметров запросов, XML-тел запросов и ответов, кодов состояния HTTP, схем данных и ограничений их полей.

Также были получены навыки работы со Swagger UI для генерации интерактивной документации по OpenAPI-спецификации, автоматической проверки корректности контракта и XML-примеров.